

BASE DE DADOS II

# Lab #02 — Bases de Dados Distribuídas

Exploração de um Banco de Dados Distribuído · **MercadoKwanza**

|                           |                                                     |
|---------------------------|-----------------------------------------------------|
| <b>Grupo</b>              | Grupo 1                                             |
| <b>Cenário / Situação</b> | Cenário A — Defensor na primeira ronda (Situação A) |
| <b>Turma</b>              | M1                                                  |
| <b>Data</b>               | 21 de Junho de 2026                                 |

ELEMENTOS DO GRUPO

|                       |                        |
|-----------------------|------------------------|
| <b>Nuno Mendes</b>    | <b>Khelv Costa</b>     |
| <b>Rafaela Mendes</b> | <b>Paula Alexandre</b> |

**Entrega:** [BDD II] Lab02 – Grupo 1 – Situação A

**Destinatário:** judson.paiva@isptec.co.ao

## 2. PARTE 1 — RESULTADOS (Fases 0 a 4)

### Fase 0 — Preparação do ambiente (Docker)

Subimos o cluster com `docker compose up -d`, que arranca quatro contentores: três nós MySQL 8.0 (Luanda, Benguela e Huambo) e um nó MongoDB 7.0. Confirmámos com `docker ps` que todos ficaram no estado `healthy`:

| NAMES        | IMAGE     | STATUS       | PORTS                    |
|--------------|-----------|--------------|--------------------------|
| no-luanda    | mysql:8.0 | Up (healthy) | 0.0.0.0:3301->3306/tcp   |
| no-benguela  | mysql:8.0 | Up (healthy) | 0.0.0.0:3307->3306/tcp   |
| no-huambo    | mysql:8.0 | Up (healthy) | 0.0.0.0:3308->3306/tcp   |
| mongo-kwanza | mongo:7.0 | Up (healthy) | 0.0.0.0:27017->27017/tcp |

Cada nó MySQL tem um `server-id` único (Luanda=1, Benguela=2, Huambo=3), requisito obrigatório para a replicação. Versão confirmada em todos: **MySQL 8.0.46**.

**Dificuldade nesta fase:** ao ligar com `-pkwanza2024` recebíamos `ERROR 1045 (28000): Access denied`, apesar de o contentor estar `healthy`. Percebemos que o `healthcheck (mysqladmin ping)` devolve sucesso mesmo com password errada e que o volume Docker antigo tinha mantido a password da primeira inicialização. Resolvemos com `docker compose down -v && docker compose up -d` para recriar os volumes de raiz.

### Fase 1 — Criação do schema `mercadokwanza`

O schema foi criado automaticamente a partir do ficheiro `dados/mercadokwanza_p1.sql`, montado em `/docker-entrypoint-initdb.d`. Ficou com 7 tabelas relacionais, ligadas por chaves estrangeiras:

| Tabela     | Descrição                                                            |
|------------|----------------------------------------------------------------------|
| PROVINCIA  | Províncias de Angola (Luanda, Benguela, Huambo)                      |
| LOJA       | Lojas, cada uma ligada a uma província ( <code>provincia_id</code> ) |
| CLIENTE    | Clientes do MercadoKwanza                                            |
| PRODUTO    | Catálogo de produtos (descrição, categoria, preço)                   |
| STOCK      | Quantidade de cada produto em cada loja                              |
| VENDA      | Cabeçalho da venda (loja, cliente, data, total)                      |
| ITEM_VENDA | Linhas de cada venda (produto, qtd, preço, desconto)                 |

### Fase 2 — Carga de dados inicial

O mesmo dataset carregou os dados base. Confirmámos as contagens em Luanda logo após a carga:

| Tabela | Registos |
|--------|----------|
|--------|----------|

|            |                                      |
|------------|--------------------------------------|
| PROVINCIA  | 3                                    |
| LOJA       | 15                                   |
| CLIENTE    | 5 000                                |
| PRODUTO    | 203 (base; passou a 253 após o P1-A) |
| STOCK      | 3 000                                |
| VENDA      | 30 001 (base; +1 na Fase 4-B)        |
| ITEM_VENDA | 89 933                               |

O dataset é determinístico (mesmos valores em qualquer máquina), o que torna os resultados reproduzíveis por qualquer elemento do grupo.

### Fase 3 — Verificação de integridade e distribuição geográfica

Validámos que as 15 lojas estão distribuídas pelas 3 províncias (5 lojas por província) e que cada loja tem o seu próprio stock. Esta distribuição geográfica é precisamente o que torna possível, mais à frente, a fragmentação horizontal de STOCK por província (Parte 2). Não foram detetadas violações de integridade referencial.

### Fase 4-B — Transação Distribuída (2PC simplificado) — `scripts/transacao.py`

Cenário testado: cliente de Luanda compra um produto cujo stock está em Benguela. O stock é decrementado em **Benguela (porta 3307)** e a venda é registada em **Luanda (porta 3301)**, com `commit` coordenado nos dois nós. Em caso de erro, `rollback` em ambos.

#### REGISTO DE RESULTADOS — FASE-4B

| Item                                                    | Valor                     |
|---------------------------------------------------------|---------------------------|
| Stock em Benguela <b>ANTES</b> da transação             | <b>153</b>                |
| Stock em Benguela <b>APÓS</b> a transação (sucesso)     | <b>143</b> (−10) ✓        |
| N.º de vendas em Luanda <b>ANTES</b>                    | <b>30001</b>              |
| N.º de vendas em Luanda <b>APÓS</b> (sucesso)           | <b>30002</b> (+1) ✓       |
| Resultado com erro forçado — <b>ROLLBACK executado?</b> | <b>S (Sim)</b>            |
| Stock em Benguela após ROLLBACK (igual ao inicial?)     | <b>143 — Sim, igual</b> ✓ |

#### Log da execução com sucesso (passo 21):

```
[Benguela] Stock decrementado: -10 unidades
[Luanda] Venda registada: id=30003
✓ Transação concluída com sucesso em ambos os nós.
```

Log da execução com erro forçado (passo 22 — INSERT da VENDA em Luanda comentado):

```
[Benguela] Stock decrementado: -10 unidades
x ROLLBACK efectuado em ambos os nós. Motivo: 1048 (23000): Column 'venda_id' cannot be null
```

**Verificação de consistência após ROLLBACK (passo 23):** Stock em Benguela voltou a **143** e o número de vendas em Luanda manteve-se em **30002** — nenhum nó ficou com alterações parciais, o que confirma a **atomicidade distribuída**.

### 3. PARTE 1 — REFLEXÕES (Fases 0 a 4)

**Sobre a Fase 4-B (transação distribuída):**

A Fase 4-B foi, para o nosso grupo, o momento em que o conceito de "transação distribuída" deixou de ser apenas teoria e passou a fazer sentido na prática. Estávamos a mexer em dois servidores MySQL diferentes ao mesmo tempo — o stock em Benguela e a venda em Luanda — e a grande questão era: como garantir que ou as duas operações acontecem, ou nenhuma acontece?

No caso de sucesso, percebemos que a transação só ficou definitivamente gravada depois de as duas fases (decremento do stock e registo da venda) terem corrido sem qualquer erro e de termos chamado o `commit()` nas duas ligações. Isto ilustrou de forma muito clara a propriedade de **atomicidade**: o princípio do "tudo ou nada". Enquanto não houvesse `commit`, nada estava realmente seguro.

O caso do erro forçado foi o mais interessante de observar. Ao comentar o INSERT da venda em Luanda, provocámos uma falha de propósito, e o programa entrou no bloco de tratamento de erro, executando `rollback()` nas duas ligações. Um pormenor que nos chamou a atenção foi o facto de a mensagem `[Benguela] Stock decrementado` ter chegado a aparecer no ecrã — mas, quando fomos verificar a base de dados, o stock estava intacto. Isto ensinou-nos que uma alteração feita dentro de uma transação ainda não confirmada é "provisória": existe apenas para aquela sessão e desaparece com o rollback. A consistência foi mantida porque nenhum dos dois nós ficou com uma alteração isolada.

Também discutimos a principal limitação desta abordagem: trata-se de um 2PC (Two-Phase Commit) **simplificado**. Se o programa Python falhasse exatamente no intervalo entre o `commit()` de Benguela e o `commit()` de Luanda, um nó ficaria confirmado e o outro não, quebrando a atomicidade. Um 2PC verdadeiro resolve isto com uma fase de *prepare* coordenada por um gestor de transações, algo que está fora do âmbito deste laboratório mas que percebemos ser essencial em produção.

### 4. REGISTO DE USO DE IA

Ferramenta utilizada: **Claude Code (modelo Claude Opus 4.8)**, via extensão no VS Code.  
Para cada interação: o pedido feito, o que a IA respondeu e o que o grupo alterou/validou.

| # | Prompt do grupo (resumo) | Resposta da IA (resumo) | O que o grupo alterou/validou |
|---|--------------------------|-------------------------|-------------------------------|
|---|--------------------------|-------------------------|-------------------------------|

|   |                                                                                                        |                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                |
|---|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1 | "Porque me dá <code>ERROR 1045 Access denied</code> ao ligar ao MySQL com <code>-pkwanza2024?</code> " | Diagnosticou que o <code>mysqladmin ping</code> do healthcheck devolve <code>exit 0</code> mesmo com password errada (logo "healthy" não prova nada), e que o volume antigo manteve a password original porque <code>MYSQL_ROOT_PASSWORD</code> só se aplica na 1ª inicialização. Sugeriu <code>docker compose down -v &amp;&amp; up -d</code> . | Validámos correndo o ping com password errada (deu <code>exit 0</code> ). Recriámos os volumes; a password passou a funcionar. |
| 2 | "Como ligo o MySQL Workbench às bases nos contentores?"                                                | Indicou host <code>127.0.0.1</code> , portas <code>3301/3307/3308</code> , user <code>root</code> , e o aviso de não usar <code>localhost</code> em Linux (socket).                                                                                                                                                                              | Usámos os parâmetros indicados para criar as 3 ligações no Workbench.                                                          |
| 3 | "Porque dá erro <code>Could not process parameters: str(...)</code> no <code>transacao.py?</code> "    | Identificou uma <b>vírgula</b> entre as duas strings do <code>INSERT INTO ITEM_VENDA</code> , que fazia o <code>execute()</code> interpretar a 2ª string como os parâmetros. Corrigiu para concatenação de strings adjacentes.                                                                                                                   | Aceitámos a correção e voltámos a correr o script com sucesso.                                                                 |
| 4 | "Executar a transação, forçar erro e registar resultados (passos 21-23)."                              | Correu o script (sucesso e erro forçado), capturou os <code>COUNT</code> antes/depois e preencheu a tabela FASE-4B.                                                                                                                                                                                                                              | Revimos os valores e confirmámos a consistência após o rollback.                                                               |
| 5 | "Gerar 50 produtos angolanos realistas e medir a propagação da replicação (P1-A)."                     | Gerou o dataset <code>dados/mercadokwanza_p2.sql</code> (50 produtos, 9 categorias) e as views de fragmentação, correu no Master e mediu a propagação e o <code>Seconds_Behind_Source</code> .                                                                                                                                                   | Revimos a lista de produtos antes de inserir; validámos os <code>COUNT</code> nos 3 nós.                                       |

|   |                                               |                                                                                                                                                          |                                                                                 |
|---|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 6 | "Escrever o script de migração para MongoDB." | Gerou <code>scripts/migracao.py</code> (PRODUTO → coleção <code>produtos</code> ; VENDA + ITEM_VENDA → coleção <code>vendas</code> com itens embebidos). | Executámos e confirmámos no MongoDB as contagens (253 produtos, 30 002 vendas). |
|---|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|

Todas as sugestões da IA foram revistas e validadas pelo grupo contra a base de dados antes de serem aceites; nenhum resultado foi assumido sem confirmação.

## 5. PARTE 2 — IMPLEMENTAÇÃO (Cenário A: Replicação + Fragmentação)

### 5.1 Arquitetura

Quatro nós em contentores Docker (ver `docker-compose.yml`):

| Nó       | Contentor                 | Papel                   | Porta host | server-id |
|----------|---------------------------|-------------------------|------------|-----------|
| Luanda   | <code>no-luanda</code>    | <b>Master</b> (escrita) | 3301       | 1         |
| Benguela | <code>no-benguela</code>  | <b>Slave / Réplica</b>  | 3307       | 2         |
| Huambo   | <code>no-huambo</code>    | <b>Slave / Réplica</b>  | 3308       | 3         |
| MongoDB  | <code>mongo-kwanza</code> | NoSQL (Parte separada)  | 27017      | —         |

O Master tem `--log-bin=mysql-bin` e `--binlog-format=ROW`. Os Slaves estão ligados ao Master via replicação assíncrona (confirmado com `SHOW REPLICATION STATUS: Replica_IO_Running=Yes, Replica_SQL_Running=Yes`).

### 5.2 Expansão do catálogo (passo 25) — `dados/mercadokwanza_p2.sql`

Inserção de **50 produtos** com nomes realistas angolanos, distribuídos por 9 categorias (Alimentação, Bebidas, Higiene, Limpeza, Electrónica, Vestuário, Papelaria, Casa, Construção). Exemplos: *Fuba de Milho Boa Safra 5kg*, *Cerveja Cuca Lata 33cl Pack 6*, *Pano Samakaka 6 Jardas*, *Saco de Cimento Nova Cimangola 50kg*. O ficheiro completo está no repositório.

A inserção é feita **apenas no Master**; a replicação propaga automaticamente para os Slaves.

### 5.3 Fragmentação horizontal de STOCK (passo 28)

```
CREATE OR REPLACE VIEW frag_stock_luanda AS
  SELECT s.* FROM STOCK s JOIN LOJA l ON s.loja_id=l.id WHERE l.provincia_id=1;
CREATE OR REPLACE VIEW frag_stock_benguela AS
  SELECT s.* FROM STOCK s JOIN LOJA l ON s.loja_id=l.id WHERE l.provincia_id=2;
```

Fragmentação **horizontal**: cada fragmento contém as *linhas* de STOCK das lojas de uma província.

---

## 6. PARTE 2 — ANÁLISE

---

### REGISTO DE RESULTADOS — P1-A

| Item                                                       | Valor                        |
|------------------------------------------------------------|------------------------------|
| COUNT PRODUTO no Luanda <b>antes</b> dos INSERTs           | <b>203</b>                   |
| COUNT PRODUTO no Benguela <b>antes</b> dos INSERTs         | <b>203</b>                   |
| COUNT PRODUTO no Benguela <b>imediatamente após</b> INSERT | <b>253</b> (≈ 119 ms depois) |
| COUNT PRODUTO no Benguela <b>após 5 segundos</b>           | <b>253</b>                   |
| <b>Seconds_Behind_Master</b> máximo observado              | <b>0</b>                     |
| Stock total fragmento <b>Luanda</b>                        | <b>262 386</b>               |
| Stock total fragmento <b>Benguela</b>                      | <b>248 892</b>               |

#### Timestamps medidos:

- T0 (instante do INSERT no Master): 21:15:40.857
- T imediato (COUNT no Benguela): 21:15:40.976 → já mostrava 253
- T+5s: 21:16:18 → 253 (estável)

#### Evidência (SHOW REPLICA STATUS):

```
Replica_IO_Running: Yes
Replica_SQL_Running: Yes
Seconds_Behind_Source: 0
Last_Error:
```

#### Análise à luz do Teorema CAP

O Teorema CAP afirma que, perante uma **partição de rede (P)**, um sistema distribuído tem de escolher entre **Consistência (C)** e **Disponibilidade (A)**.

- A replicação **Master-Slave assíncrona** do MySQL privilegia **Disponibilidade (AP)**: os Slaves continuam a responder a leituras mesmo que o Master fique inacessível, mas podem servir dados **ligeiramente desatualizados** (consistência eventual). No nosso teste o atraso foi ~0, mas sob carga elevada ou rede degradada o `Seconds_Behind_Master` aumentaria.
  - A transação 2PC da Fase 4-B, pelo contrário, privilegia **Consistência (CP)**: exige que ambos os nós confirmem antes de validar; se um falhar, tudo é revertido — ao custo de bloquear/abortar (menor disponibilidade).
  - Conclusão: o MercadoKwanza usa **ambas as estratégias conforme a necessidade** — replicação AP para o catálogo (leituras rápidas, tolerância a atraso) e transações CP para operações críticas de stock/venda (não pode haver dinheiro/stock inconsistente).
-

## 7. PARTE 2 — REFLEXÕES (individuais)

---

### 1. O Slave recebeu os dados antes ou depois dos 5 segundos? O que influencia este tempo de propagação?

No nosso teste, o Slave recebeu os 50 produtos quase imediatamente: medimos uma diferença de apenas cerca de 119 milésimos de segundo entre o instante do INSERT no Master e o momento em que o COUNT em Benguela já mostrava 253 produtos. Ou seja, a propagação aconteceu muito **antes** dos 5 segundos — quando voltámos a verificar aos 5 segundos, o valor já era exatamente o mesmo, e o `Seconds_Behind_Master` manteve-se sempre em 0.

Percebemos que este tempo de propagação não é fixo e depende de vários fatores. Os principais são: o volume e o tamanho das transações inseridas (50 linhas é muito pouco), a latência da rede entre os nós (aqui é uma rede Docker local, praticamente sem atraso), a carga de trabalho que o Master tem no momento, o formato do binlog (usámos ROW) e, sobretudo, a rapidez com que a thread SQL do Slave consegue ler e aplicar o relay log. Num cenário real, com servidores em províncias diferentes, ligações de internet mais lentas e milhares de operações por segundo, este atraso seria seguramente maior e mais visível.

### 2. Se neste momento o Master caísse, os Slaves teriam todos os dados? O que poderia ter ficado por replicar?

Não necessariamente. Como a replicação que usámos é **assíncrona**, o Master confirma a escrita ao cliente sem esperar que os Slaves a tenham recebido. Isto significa que, se o Master falhasse de forma abrupta, qualquer transação que já tivesse sido confirmada no Master mas ainda não tivesse sido transferida e aplicada pelos Slaves ficaria **perdida**. O tamanho desta "janela de perda" corresponde, na prática, ao valor de `Seconds_Behind_Master` no momento da falha.

No nosso caso concreto, como o atraso era 0, a perda seria mínima ou nula. No entanto, percebemos que isto foi sorte do cenário (pouca carga e rede local) e não uma garantia. Para reduzir este risco existiria a replicação **semi-síncrona**, em que o Master só confirma a escrita depois de pelo menos um Slave acusar a receção dos dados — mais seguro, mas mais lento.

### 3. A fragmentação de STOCK por província faz sentido para o negócio do MercadoKwanza? Que consultas seriam mais rápidas e quais seriam mais lentas?

Sim, faz sentido, porque o stock é, por natureza, um dado geograficamente local: cada província gere as suas próprias lojas e o seu próprio inventário. Ao fragmentar a tabela STOCK por `provincia_id`, cada região passa a trabalhar essencialmente sobre o seu fragmento, o que reflete bem a forma como o negócio funciona na realidade.

As consultas que ficariam **mais rápidas** são as locais — por exemplo, "qual o stock disponível nas lojas de Luanda" — porque cada fragmento contém apenas as linhas daquela província e o motor lê muito menos dados. Em contrapartida, as consultas que ficariam **mais lentas** são as globais, que precisam de olhar para todo o país — por exemplo, "qual o stock total do produto X em Angola inteira" — porque obrigam a juntar (com `UNION` ou agregação) os resultados de todos os fragmentos. Concluimos, por isso, que a fragmentação é uma boa decisão se a maioria das consultas do dia a dia for regional, que é exatamente o caso de uma cadeia de supermercados como o MercadoKwanza.

---

## 8. DIFICULDADES

---



1. `Access denied` **no MySQL apesar da password correta no compose**. Diagnóstico: o healthcheck com `mysqladmin ping` ficava "healthy" mesmo com password errada (devolve exit 0 se o servidor responder), e o volume Docker antigo tinha mantido a password da primeira inicialização. Resolução: `docker compose down -v` para recriar os volumes do zero.
2. `Could not process parameters: str(...)` **no transacao.py**. Diagnóstico: uma vírgula a separar as duas strings do `INSERT INTO ITEM_VENDA` fazia o conector tratar a 2ª string como os parâmetros. Resolução: remover a vírgula (strings adjacentes concatenam em Python).

---

## 9. CONCLUSÃO

---

Neste laboratório montámos um sistema de base de dados distribuído real, com um servidor principal em Luanda e duas réplicas em Benguela e Huambo. Aprendemos, na prática, que **replicar dados não é instantâneo nem garantido**: funciona muito bem quando tudo está saudável (a cópia chegou em milésimas de segundo), mas se o servidor principal falhar no momento errado pode perder-se informação. Também percebemos que uma transação que mexe em dois sítios ao mesmo tempo tem de ser "tudo ou nada" — e vimos isso a acontecer quando forçámos um erro e tudo foi revertido. Por fim, dividir o stock por província (fragmentação) torna as consultas locais mais rápidas, mas as consultas que olham para o país inteiro ficam mais pesadas. No fundo, não há solução perfeita: escolhe-se entre rapidez/disponibilidade e consistência consoante o que o negócio precisa.

---

## 10. GITHUB

---

- **Repositório:**

<https://github.com/nunoom/BDII-Lab02> Explora-o de Banco de Dados Distribu-dos

- O repositório (público) inclui: `docker-compose.yml`, os datasets em `dados/` (`mercadokwanza_p1.sql` e `mercadokwanza_p2.sql`), os scripts/ (`transacao.py`, `transacao_erro.py`, `migracao.py`, `replicacao.sql`), o relatório em `relatorio/` e um `README.md` explicativo com a arquitetura, as portas de cada nó e instruções de execução.

---

## 11. REFERÊNCIAS (APA 7.ª edição)

---

1. Brewer, E. (2012). CAP twelve years later: How the "rules" have changed. *Computer*, 45(2), 23–29. <https://doi.org/10.1109/MC.2012.37>
2. Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59. <https://doi.org/10.1145/564585.564601>
3. Özsu, M. T., & Valduriez, P. (2020). *Principles of distributed database systems* (4th ed.). Springer. <https://doi.org/10.1007/978-3-030-26253-2>
4. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.
5. Tanenbaum, A. S., & van Steen, M. (2017). *Distributed systems* (3rd ed.). Maarten van Steen.
6. Oracle Corporation. (2024). *MySQL 8.0 reference manual — Replication*. <https://dev.mysql.com/doc/refman/8.0/en/replication.html>